

MANUALE TECNICO - CINEMAX

Margiotta Alessandro
Arpini Daniele
Redondo Botia Juan Andres

Giugno 2026

Versione 1.0

Corso di laurea in informatica
Laboratorio Interdisciplinare A

1. INTRODUZIONE	3
2. INSTALLAZIONE	4
1.1 Requisiti di sistema e Setup ambiente	4
2.2 Installazione del programma	4
3. ARCHITETTURA E STRUTTURA DEL PROGETTO	4
3.1 Organizzazione dei package	4
3.2 Organizzazione dei file di persistenza	5
3.3 Analisi delle classi della cartella "Model"	5
3.4 Analisi classe GestoreFile	6
3.5 Analisi classe main	6
3.6 Scelte delle strutture dati	7
4. ESECUZIONE ED USO: ANALISI DEI MENU	7
4.1 Menu principale	7
4.2 Menu guest	7
4.3 Menu cliente	8
4.4 Menu proiezionista	8
4.5 Menu bigliettotaio	8
4.6 Data set di test	8
Proiezioni.csv:	9
5. LIMITI DELLA SOLUZIONE	10
6. SITOGRAFIA / BIBLIOGRAFIA	10

1.INTRODUZIONE

Questo manuale tecnico descrive l'architettura logica, la struttura delle classi e le scelte di implementazione per lo sviluppo della applicazione CineMax.

Il documento è rivolto agli sviluppatori e a tutti coloro che saranno responsabili della manutenzione, dei test e la futura estensione del sistema.

Il sistema è stato implementato in linguaggio Java dove l'obiettivo principale è automatizzare le operazioni quotidiane di un cinema monosala, eliminando la necessità di registrare i dati manualmente e riducendo il rischio di errori nella gestione degli orari e dei posti.

Per coprire tutte le necessità di un cinema, questa applicazione offre strumenti specifici per tre categorie distinte di utenti, ognuna con un menu dedicato e permessi differenti:

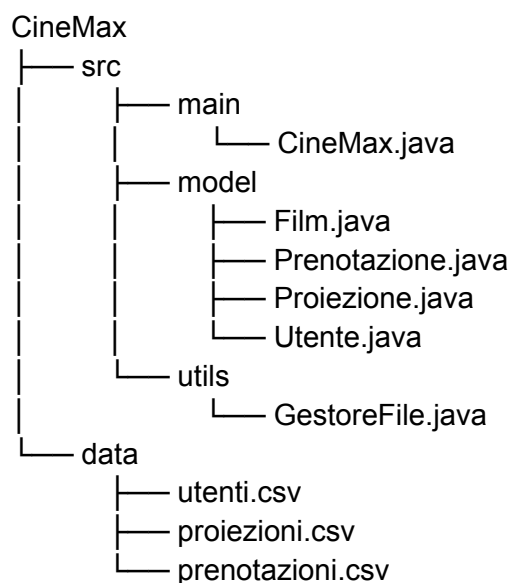
- Clienti: Visualizzare film, cercare le proiezioni e prenotare biglietti.
- Bigliettai: Visualizzare tutte le prenotazioni della giornata e cercarne una specificamente
- Proiezionisti: Responsabili della pianificazione del palinsesto, dell'inserimento dei nuovi film e del controllo degli orari per evitare sovrapposizioni in sala.

2.INSTALLAZIONE

1.1 Requisiti di sistema e Setup ambiente

Per poter eseguire il programma CineMax, il computer deve soddisfare pochi requisiti:

- JDK: Si richiede la versione 11 o successiva, perché abbiamo utilizzato funzioni più moderne di Java come gli stream e le classi per gestire le date che non funzionano sulle versioni vecchie di Java.
- Console di comando: Il programma funziona senza alcuna interfaccia grafica quindi userà solo il prompt dei comandi su windows o il terminale su Mac/Linux.
- Organizzazione delle cartelle: Il progetto è organizzato della seguente maniera:



2.2 Installazione del programma

Il programma non ha bisogno di installatori particolari. Per compilarlo ed avviarlo bastano i comandi standard di Java eseguiti dal terminale, posizionandosi dentro la cartella principale del progetto. Al primo avvio, il programma si occupa anche di creare i file per salvare i dati se non sono già presenti sul computer.

3. ARCHITETTURA E STRUTTURA DEL PROGETTO

3.1 Organizzazione dei package

Il codice sorgente dell'applicazione CineMax è strutturato in tre package diversi all'interno della cartella "src".

- Package Main: Contiene la classe principale "CineMax". Inizializza i dati, gestisce l'input della tastiera mediante uno Scanner e controlla i menu testuali per i quattro stati di accesso dell'applicazione (Menu principale, guest, cliente, proiezionista e biglietto).
- Package model: contiene le quattro classi che definiscono gli oggetti del sistema:
 - Film: Che memorizza i dati dei film.
 - Proiezione: Associa un film a una data/ora e a un prezzo, impostando la capienza massima della sala a 200 posti.
 - Utente: Gestisce le anagrafiche e i ruoli di accesso, i quali sono: cliente, bigliettaio, proiezionista.
 - Prenotazione: Registra i posti riservati dai clienti identificandoli con un codice univoco generato tramite UUID.
- Package utils: Contiene la classe di servizio GestoreFile che ci pensa esclusivamente alla logica tecnica, come la lettura/scrittura dei file e la cifratura delle password tramite l'algoritmo SHA-256.

3.2 Organizzazione dei file di persistenza

Per evitare la perdita dei dati alla chiusura del programma, l'applicazione salva tutte le informazioni in formato CSV all'interno della cartella "data" nella radice del progetto. I file sono:

- Utenti.csv: Contiene i campi "nome", "cognome", "username", "passwordHash", "dataNascita", "domicilio" e "ruolo". All'avvio del sistema, se il file non esiste, viene creato automaticamente con 2 proiezionisti e 5 bigliettai predefiniti.
- Proiezioni.csv: Memorizza il palinsesto del cinema con i dettagli del film associato, i campi sono: "id", "titolo", "genere", "regista", "anno", "durata", "etaMinima", "dataOra", "costo".
- prenotazioni.csv: Salva tutte le prenotazioni confermate registrando il codice UUID, lo username e il nome del cliente, l'ID della proiezione e il numero di posti acquistati.

3.3 Analisi delle classi della cartella "Model"

Le classi dentro del pacchetto model servono a definire come sono fatti i dati dentro del nostro programma. Dentro di ogni classe si fa uso dell'incapsulamento:

- Classe Film: Rappresenta un film ed ha come attributi il titolo, genere, regista, l'anno, la durata in minuti e l'età minima.
- Classe Utente: Rappresenta a chi usa il sistema. I suoi attributi sono nome, cognome, username, password (cifrata), oltre agli attributi normali ci sono anche gli attributi "FINAL" che sono il tipo di cliente che usa il sistema (proiezionista, cliente, bigliettaio).
Utilizza i metodi come isCliente() per capire il ruolo della persona che sta provando a usare il sistema.
- Classe Proiezione: Rappresenta una proiezione specifica. Collega un oggetto "Film" a una data e ora e a un costo del biglietto. Contiene anche la costante "Capacità sala" per stabilire il limite dei posti.
- Classe prenotazione: Rappresenta i posti bloccati da un cliente. Ha la particolarità di avere due costruttori, uno genera un codice automatico e univoco usando la libreria UUID di Java quando un cliente fa una nuova prenotazione, l'altro riceve il codice direttamente come stringa quando dobbiamo ricaricare le prenotazioni vecchie dal file CSV.

3.4 Analisi classe GestoreFile

si occupa interamente della logica di persistenza dei dati e di alcune funzioni di utilità che servono a supportare l'intero programma. È una classe composta esclusivamente da metodi statici, il che significa che non ha bisogno di essere istanziata per essere usata.

- Flussi input/output: Per la lettura dei file CSV le classi BufferedReader e InputStreamReader. Per la scrittura si usa PrintWriter e OutputStreamWriter. Questa combinazione è stata scelta perché permette di specificare esplicitamente la codifica dei caratteri UTF-8 facendo che il file venga letto su qualsiasi sistema operativo.
- Lettura: Nei metodi di caricamento come "caricaProiezioni". il file viene letto riga per riga. Ogni riga viene spezzata in un array di stringhe usando il metodo ".split(;)". Il codice controlla che l'array contenga il numero corretto di campi (es. almeno 9 campi per le proiezioni e 7 per gli utenti) per il corretto funzionamento dei file CSV usando "if (campi.length < 9 oppure 7) continue" saltando le righe corrotte o non valide senza far crashare il programma.

Questa classe implementa anche il metodo hashSHA256 tramite la classe nativa di Java MessageDigest. Si usa per garantire la sicurezza dei clienti perché con questo metodo le password dei clienti non vengono mai salvate in modo chiaro nei file "utenti.csv", invece su questo file le password vengono salvate in un modo illeggibile.

3.5 Analisi classe main

è l'architettura di controllo centrale dell'intera applicazione. Ha il compito di coordinare l'avvio, mantenere i dati in memoria durante l'uso e gestire l'interazione con l'utente.

- Mantiene in memoria le liste globali (proiezioni, utenti, prenotazioni) che fungono da tabelle temporanee del database durante l'esecuzione. Mantiene inoltre il riferimento all'oggetto `utenteCorrente` (di tipo `Utente`), che funge da sessione attiva: se è "null" il programma sa che l'utente sta navigando come guest, altrimenti contiene l'oggetto dell'utente loggato con il suo specifico ruolo.
- La classe istanzia un unico oggetto `Scanner`. Questa scelta evita conflitti di lettura sul canale di input standard che potrebbero verificarsi creando e chiudendo più oggetti `Scanner` all'interno dei singoli metodi.
- Il flusso del programma non è un unico blocco di codice, ma è diviso in sotto-metodi gerarchici (`menuPrincipale`, `menuCliente`, `menuProiezionista`, ecc.). Lo smistamento delle chiamate avviene tramite costrutti `switch` inseriti in cicli `while`, dai quali si esce solo tramite istruzioni di `return` esplicite (es. al momento del `logout` o della chiusura).

3.6 Scelte delle strutture dati

Quando il programma è in esecuzione, tutti i dati letti dai file CSV vengono caricati e gestiti all'interno di oggetti di tipo `List`, implementati tramite la classe `ArrayList` (es.

`List<Proiezione>`, `List<Utente>`...).

Abbiamo scelto `ArrayList` perché:

- È la struttura dati più dinamica e flessibile di Java per gestire elenchi di elementi che possono aumentare o diminuire (ad esempio quando si aggiunge o cancella una proiezione)
- Permette di scorrere gli elementi in modo molto rapido.

Dato che i dati sono salvati in una lista sequenziale, quando cerchiamo un film o un utente specifico dobbiamo scorrere la lista dall'inizio alla fine. Questa operazione ha una complessità di $O(n)$.

4. ESECUZIONE ED USO: ANALISI DEI MENU

4.1 Menu principale

Il controllo della navigazione tra i menu si basa su cicli `while` e costrutti `switch-case` che intercettano l'input numerico digitato dall'utente sul terminale. All'interno di questa sezione:

- `login`: Gestisce il processo di autenticazione dell'utente all'interno del sistema.
 - Il metodo richiede l'inserimento di `username` e `password` tramite la console. Subito dopo, invoca la funzione di hashing per trasformare la password inserita in un codice SHA-256. A questo punto, esegue una ricerca sequenziale all'interno della lista globale degli utenti e trova un record in cui coincidono sia lo `username` sia l'hash della password, l'autenticazione va a

buon fine, il metodo restituisce un valore booleano positivo e l'oggetto utente viene salvato nella sessione corrente del programma.

- **registraCliente:** Permette a un nuovo utente di inserire i propri dati nel sistema per creare un profilo di tipo "cliente".
 - scorre l'intera lista degli utenti esistenti per verificare che lo username scelto non sia già stato occupato. Se lo username è disponibile, il sistema applica l'algoritmo SHA-256 alla password inserita, istanzia un nuovo oggetto della classe utente e richiama la funzione di scrittura della classe GestoreFile per aggiornare immediatamente il file CSV.

4.2 Menu guest

Permette esclusivamente di visionare la programmazione dei film.

- **menuguest:** Richiede l'inserimento di un titolo (anche parziale). Il programma prende la lista globale delle proiezioni e la trasforma in uno Stream. Utilizza il metodo `.filter()` per verificare se il titolo del film contiene la stringa inserita dall'utente. I risultati filtrati vengono poi raccolti in una nuova lista dinamica tramite `.collect(Collectors.toList)`. Se la stringa inserita è vuota, il filtro fa passare tutte le proiezioni, se si trovano match, questi vengono stampati alla fine.

4.3 Menu cliente

Rappresenta l'interfaccia operativa per gli utenti che sono già registrati, offre la funzionalità di gestione, prenotazione e modifica di posti.

- Funziona tramite un ciclo while e una gestione switch-case, smista le scelte verso i metodi specifici del dominio cliente per la consultazione del palinsesto, la creazione di una nuova prenotazione, la lettura dei propri biglietti, la modifica o l'eliminazione. La scelta di uscita azzera la sessione richiamando il metodo `logout()`.

4.4 Menu proiezionista

Fa da pannello di controllo tecnico per la gestione del palinsesto dei film e degli orari del cinema. Mette a disposizione al personale tecnico gli strumenti di amministrazione delle proiezioni.

- Usa la medesima struttura del menu cliente. Usa le quattro operazioni fondamentali di amministrazione dei dati, la stampa totale della lista in memoria, l'inserimento di un nuovo spettacolo, la modifica dei dati di una proiezione esistente o la sua cancellazione definitiva.

4.5 Menu bigliettaio

Rappresenta l'interfaccia semplificata per il personale che è presente nelle casse del cinema. Consente un controllo rapido e mirato sulle prenotazioni per l'emissione dei biglietti.

- Gestisce un menu ristretto focalizzato su due sole operazioni operative: il filtraggio automatico delle prenotazioni associate alla data corrente del computer e la ricerca puntuale di una determinata transazione.

4.6 Data set di test

Per garantire la persistenza dei dati senza dipendere da sistemi esterni, l'applicazione utilizza file di testo in formato CSV posizionati nella directory "data". I dati all'interno di questi file sono strutturati in righe (record), dove ogni riga rappresenta un oggetto e i singoli attributi sono separati dal carattere punto e coma.

- Struttura e Campi dei File:
 1. Utenti.csv: Memorizza le credenziali e le informazioni anagrafiche di tutte le persone registrate nel sistema.
 - Campi:
nome;cognome;username;passwordHash;dataNascita;domicilio;ruolo.
La password è memorizzata esclusivamente come hash SHA-256 e il ruolo può essere cliente, proiezionista e bigliettaio.
 2. Proiezioni.csv: Contiene la programmazione dei film all'interno del cinema. Ogni riga include sia i dettagli della proiezione sia la scheda tecnica del film associato.
 - Campi: id;titolo;genere;regista;anno;durata;etaMinima;dataOra;costo.
L'ID della proiezione segue il formato "P" seguito dal timestamp.
 3. Prenotazioni.csv: Registra le transazioni e i posti occupati dai clienti per ciascuno spettacolo.
 - Campi:
codice;usernameCliente;nomeCliente;idProiezione;numeroPosti. Il codice della prenotazione è un identificativo alfanumerico univoco generato dall'algoritmo UUID.

Utenti.csv:

NOME E COGNOME	USERNAME	PASSWORD	RUOLO
Juan Redondo	JuAndres	1234	Cliente
Laura Bianchi	lbianchi	password123	Proiezionista
Luca Neri	lneri	password123	Bigliettaio

Proiezioni.csv:

ID	Titolo	Genere	Regista	Anno	Durata	Età Minima	Data / Ora	Costo
P1781265149540	Titanic	Drama	James Cameron	1998	194	16	30/06/2026	8,50

Prenotazioni.csv

codice	usernameCliente	nomeCliente	idProiezione	numeroPosti
21bbc62e-e657-492b-b75c-b456939b02b3	JuAndres	Juan Redondo	P1781265149540	5

5. LIMITI DELLA SOLUZIONE

L'applicazione *CineMax* soddisfa pienamente tutti i requisiti funzionali richiesti per la gestione del cinema. Tuttavia, l'architettura software adottata presenta alcuni limiti strutturali ed algoritmici legati alle scelte tecnologiche di partenza.

- **Complessità Computazionale Lineare delle Ricerche:**
L'utilizzo delle strutture dati in memoria di tipo ArrayList implica che ogni operazione di ricerca (come trovare un utente per il login, un film specifico o una prenotazione) venga eseguita scorrendo sequenzialmente la lista dall'inizio alla fine. Questa operazione ha una complessità pari a $O(n)$. Sebbene per il set di dati di un singolo cinema l'operazione risulti istantanea, se il sistema dovesse gestire milioni di record, le performance subirebbero un forte rallentamento
- **Limitazioni dell'Interfaccia Utente:**
L'interfaccia a riga di comando, sebbene leggera e priva di dipendenze, limita l'esperienza d'uso.
- **Singola Sala e Capienza Rigida:**
L'architettura attuale del software presuppone che il cinema sia composto da un'unica sala cinematografica condivisa per tutte le proiezioni dove la costante

“CAPACITA_SALA = 200” è applicata in modo globale e rigido a qualsiasi film inserito nel palinsesto. Questa scelta strutturale impedisce di gestire contesti reali come i cinema multisala, dove film diversi possono essere proiettati contemporaneamente in stanze differenti con capienze variabili.

6. SITOGRAFIA / BIBLIOGRAFIA

- Oracle Java SE Documentation
<https://docs.oracle.com/en/java/>
- Google docs
Creazione dei manuali del progetto

